

DNN Mapping Optimization on Spatial Accelerators: Problem Formulation and Algorithm Comparison

Author Name

Course Project: Integrated Circuit Engineering Algorithms

Code: <https://github.com/mcyinsz/EDA-alg-spatial-mapping>

Abstract—Spatial accelerators (e.g., Cerebras WSE, Groq TSP) interconnect thousands of compute-memory cores in a 2D mesh. Deploying a DNN requires solving a coupled partitioning (cores per layer) and placement (physical mesh positions) problem whose search space grows combinatorially. We formulate inference-latency minimization with an analytical cost model covering compute parallelism, inter-layer communication, and intra-layer tensor-parallel synchronization. Four solvers—integer linear programming (ILP, used as a placement reference), simulated annealing (SA), a nested evolution strategy (EA), and Greedy+Kernighan-Lin (Greedy+KL)—are compared on 19 feasible configurations (7 workloads $\times$ 3 mesh sizes). After correcting the experimental setup so that solvers may leave cores idle and use cost-aware partitioning, Greedy+KL wins 12/19, SA 6/19, and EA 1/19 under the main model. An inter-only ablation yields SA 8/19 and Greedy+KL 8/19; a κ sweep and an intra-parallel ablation further reshuffle the ranking (EA reaching 9/19). The central finding is that modeling assumptions and experimental setup jointly and decisively determine algorithm rankings, a caution for analytical mapping studies.

Index Terms—spatial accelerator, neural network mapping, combinatorial optimization, simulated annealing, evolutionary algorithm

I. Introduction

Spatial architectures place compute and memory together on hundreds to millions of cores connected by a 2D mesh network-on-chip (NoC). Representative platforms include the Cerebras WSE-3 ($\sim 900k$ cores) [7], the Groq TSP [8], and Intel Loihi 2. On such hardware, inference latency depends not only on what model runs but on how it is mapped: a poor mapping causes either load imbalance (compute bottleneck) or NoC congestion (communication bottleneck) [1], [2].

The mapping problem couples two sub-problems. Partitioning decides how many cores each layer receives, trading parallelism for communication volume. Placement decides where those cores sit, affecting Manhattan routing distance. The two are tightly coupled: changing the partition alters the optimal placement and vice versa. For a 10-layer network on a 152-core chip, partitioning alone exceeds 10^{13} choices and placement exceeds 10^{267} [3].

Prior mapping tools such as Timeloop [4], GAMMA [5], and DiGAMMA [6] target hierarchical-memory accelerators and model tiling and dataflow ordering rather than NoC routing distance and physical placement. Wever et al. [3] introduce evolutionary mapping with hardware-in-the-loop on Loihi 2, but require ~ 5 s per evaluation on a

single 152-core chip. We instead study a general analytical formulation and contribute: (1) a unified three-term cost model for heterogeneous (linear and convolutional) layers; (2) four solvers in a common framework with a corrected, fair setup (idle cores allowed, aligned budgets, ILP repositioned as a placement reference); and (3) systematic ablations (κ sweep, intra-parallel assumption, inter-only) showing that modeling choices dominate the observed algorithm ranking.

II. Problem Formulation

A. Model and Variables

A workload has L layers; layer i is described by FLOPs F_i , weight bytes W_i , and output-activation bytes A_i . For a linear layer with input/output dims n_{i-1}, n_i and sparsity s_i : $F_i = 2n_{i-1}n_i(1-s_i)$, $W_i = 4n_{i-1}n_i(1-s_i)$, $A_i = 4n_i$; convolutional layers use the analogous $C_{in}, C_{out}, k, H_{out}, W_{out}$ expressions. The accelerator is an $H \times W$ mesh of $K=HW$ homogeneous cores with per-core memory M , throughput P (FLOP/cycle), and link cost β (cycles per byte-hop). The decision variables are the partition $\mathbf{x} = (x_1, \dots, x_L) \in \mathbb{Z}_+^L$ and the placement sets $S_i \subset \{(r, c)\}$ with $|S_i| = x_i$.

B. Objective

We minimize single-inference latency (in cycles):

$$\min_{\mathbf{x}, \{S_i\}} T = \sum_{i=1}^L T_{\text{comp},i} + \sum_{i=1}^{L-1} T_{\text{inter},i} + \sum_{i=1}^L T_{\text{intra},i} \quad (1)$$

with the three terms

$$T_{\text{comp},i} = \frac{F_i}{x_i P}, \quad T_{\text{inter},i} = \beta \frac{A_i}{x_i} \bar{d}_{\text{inter}}(S_i, S_{i+1}), \quad (2)$$

$$T_{\text{intra},i} = \beta \kappa_i A_i \left(1 - \frac{1}{x_i}\right) \bar{d}_{\text{intra}}(S_i) \quad (x_i > 1), \quad (3)$$

where $\bar{d}_{\text{inter}}$ and $\bar{d}_{\text{intra}}$ are the average pairwise Manhattan distances between and within layer core sets, and κ_i is the tensor-parallel factor ($\kappa_i=1.0$ for linear, 0.5 for convolution). Equation (3) models a simplified 1-D tensor-parallel synchronization: each of x_i cores holds an A_i/x_i shard and exchanges data with the others, giving volume of order $A_i(1-1/x_i)$.

C. Constraints and Properties

(C1) $S_i \cap S_j = \emptyset$ for $i \neq j$; (C2) $(W_i + A_i)/x_i \leq M$; (C3) $x_i \geq 1$; and $\sum_i x_i \leq K$, where idle cores are explicitly allowed ($\sum_i x_i$ may be strictly less than K). From (C2), $x_i^{\min} = \lceil (W_i + A_i)/M \rceil$. The objective is non-convex (piecewise-constant distance terms), tightly coupled ($\mathbf{x}$ appears in both numerator and denominator), and combinatorially explosive.

III. Algorithms

ILP (placement reference). Binary $z_{i,k}$ assign core k to layer i ; quadratic distance terms are linearized with auxiliary variables $u_{i,k,l}$ (inter) and $w_{i,k,l}$ (intra), totaling $O(K^2L)$. Because this scale is prohibitive, the ILP fixes a greedy partition and optimizes placement only. We therefore treat it as a reference for the placement subproblem rather than a joint optimizer; it runs with a 60 s limit on ≤ 16 -core meshes (PuLP+CBC).

Simulated Annealing (SA). State $(\mathbf{x}, \{S_i\})$. The partition neighborhood now offers three moves—transfer a core between layers, discard a core (it becomes idle), and enable an idle core—and the placement neighborhood swaps $\lfloor \sqrt{K_{\text{used}}} \rfloor$ core pairs. The initial state uses the minimum per-layer cores (no forced filling). Metropolis acceptance with $T_0=100$, $\gamma=0.995$, 3000 iterations, 5 runs.

Nested Evolution Strategy (EA). Following [3]: an outer ES evolves a partition genotype ($x_i^{\text{extra}}, C_{\text{unused}}$) and an inner ES evolves a placement permutation, with a reordering operator that preserves spatial locality when the partition changes. $(1+\lambda)$ selection, $\lambda=4$, 40 generations, 5 runs.

Greedy+KL. (1) cost-aware greedy partitioning that adds a core only when total latency (including communication) decreases; (2) column-major contiguous placement followed by Kernighan-Lin pair swaps; (3) partition refinement that may move or remove a core. Deterministic, one run.

IV. Experimental Setup

Tables I and II list the seven workloads and solver parameters. All meshes use $M=512$ KB, $P=64$ FLOP/cycle, $\beta=1.0$ at 1 GHz; latencies are in μs and only meaningful relatively. Large-MLP and Transformer-L exceed the 16 cores of the 4×4 mesh, leaving 19 feasible configurations. Five baselines (random, packed row/col-major, spread, equal-partition) are included. We run three ablations: – inter-only (drop T_{intra}), – intra-parallel (divide T_{intra} by x_i , matching the inter-layer parallel-injection assumption), and a global κ sweep (scripts/sweep_kappa.py). All solver outputs are re-verified by recomputing latency from the stored placements (928/928 checks pass).

V. Results and Analysis

A. Main Ranking

Table III reports the best latency per solver. Under the main model Greedy+KL wins 12/19, SA 6/19, EA 1/19. The wins split by family as Greedy+KL leading on the

TABLE I: DNN workloads.

Name	Structure	L	FLOPs
Small-MLP	MLP [128,256,128,64]	3	147K
Medium-MLP	MLP [256,512,1024,512,256]	4	2.62M
Large-MLP	MLP [256,512,1024,1024,512,256]	5	4.72M
Sparse-MLP	Medium-MLP, 50% sparse	4	1.31M
Transformer-S	2×Encoder, $d=128$	12	786K
Transformer-L	2×Encoder, $d=256$	12	3.15M
ConvNet	3×Conv2D + FC	4	85.3M

TABLE II: Solver configuration and mean compute budget (main experiment).

Solver	Key parameters	Runs	Evals/run
ILP	60 s, gap 0.05, ≤ 16 cores	1	1
SA	$T_0=100$, $\gamma=0.995$, 3000 it	5	~ 3002
EA	$\lambda=4$, 40 generations	5	~ 322
Greedy+KL	KL passes 8, refine 15	1	~ 331

larger MLPs, Transformers, and ConvNet/6×6, while SA takes all three Small-MLP meshes (a flat 3.20 μs) and the compute-bound ConvNet/8×8. The Small-MLP result is the clearest effect of the setup fix: by keeping cores idle, SA avoids the spurious intra-layer cost that previously made optimizers lose to a trivial packed baseline.

B. Why Greedy+KL Leads on Large Workloads

Greedy+KL’s contiguous, column-major placement keeps intra-layer average distance near 1–2 hops, whereas SA’s random swaps and EA’s mutations fragment a layer’s cores (3–5 hops). On the 8×8 mesh, where distances reach 14 hops, this gap widens: on Transformer-L/8×8 Greedy+KL (54.4 μs) beats SA (68.2) by 20% and EA (74.6) by 27%. The compute-/inter-/intra-layer breakdown (Fig. 3) confirms Greedy+KL keeps the intra-layer share lowest. Its cost-aware partitioning also avoids overprovisioning on tiny workloads, fixing the previous degradation where forced core-filling inflated communication.

C. Where the Heuristics Differ

SA’s idle-core moves dominate Small-MLP (3.20 vs. Greedy 3.46 μs) and remain competitive on the compute-bound ConvNet/8×8. EA is limited by its small evaluation budget (~ 322 evals/run, an order of magnitude below SA), winning only Transformer-S/4×4 in the main model. Run-to-run variation is modest for the deterministic regimes but appreciable for ConvNet/6×6 (SA mean $1132 \pm 189 \mu\text{s}$) where the random-swap neighborhood struggles, reinforcing that the best-of-5 numbers favor the stochastic solvers.

D. Ablation 1: Inter-only Model

Removing the intra-layer term re-balances the ranking to SA 8, Greedy+KL 8, EA 3. Without a compactness incentive, Greedy+KL’s KL swaps can fragment layers badly (ConvNet/8×8 1005 μs), while EA wins all three Transformer-S meshes.

TABLE III: Best inference latency (μs), main model. Bold = best joint optimizer (SA/EA/Greedy+KL) per config; ILP is a placement reference.

Configuration	Baseline	ILP	SA	EA	Greedy+KL
Small-MLP/4 $\times$ 4	3.84	4.66	3.20	3.33	3.58
Small-MLP/6 $\times$ 6	3.84	—	3.20	4.98	3.46
Small-MLP/8 $\times$ 8	3.84	—	3.20	7.63	3.46
Medium-MLP/4 $\times$ 4	27.1	26.9	25.0	25.5	25.2
Medium-MLP/6 $\times$ 6	29.3	—	31.5	30.2	26.1
Medium-MLP/8 $\times$ 8	32.3	—	36.1	41.0	28.1
Large-MLP/6 $\times$ 6	44.7	—	44.7	47.3	40.0
Large-MLP/8 $\times$ 8	47.2	—	52.2	59.4	42.7
Sparse-MLP/4 $\times$ 4	21.8	22.3	20.6	20.9	21.0
Sparse-MLP/6 $\times$ 6	26.2	—	23.6	29.3	22.3
Sparse-MLP/8 $\times$ 8	24.6	—	26.2	39.8	24.6
Transformer-S/4 $\times$ 4	23.2	21.4	21.8	21.3	22.5
Transformer-S/6 $\times$ 6	21.9	—	25.2	27.0	23.6
Transformer-S/8 $\times$ 8	24.6	—	29.2	33.5	21.9
Transformer-L/6 $\times$ 6	57.7	—	62.6	61.8	57.0
Transformer-L/8 $\times$ 8	68.7	—	68.2	74.6	54.4
ConvNet/4 $\times$ 4	859	826	830	816	811
ConvNet/6 $\times$ 6	838	—	951	889	782
ConvNet/8 $\times$ 8	937	—	1032	1070	1166

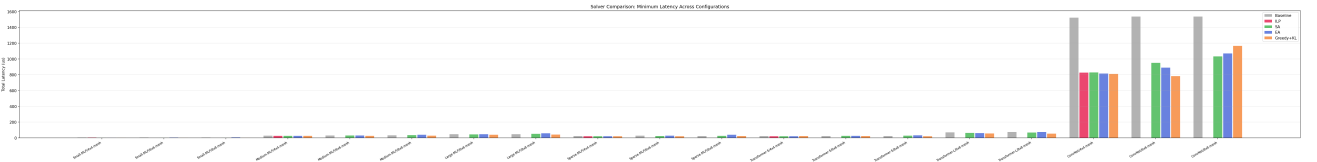
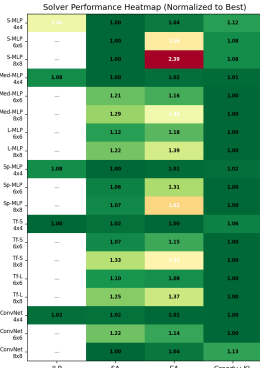
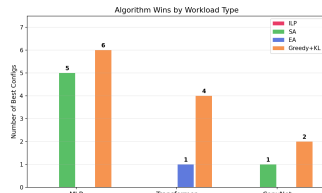


Fig. 1: Best inference latency per solver across all 19 configurations (log-scale y-axis for ConvNet).



(a) Normalized latency.



(b) Wins by network type.

Fig. 2: Relative performance heatmap and per-family win counts (main model).

TABLE IV: Solver wins vs. κ (19 configs).

	$\kappa=0$	0.25	0.5	1.0	2.0
SA	8	4	4	6	6
EA	3	11	6	1	2
Greedy+KL	8	4	9	12	11

TABLE V: Greedy+KL core usage on Large-MLP/8 $\times$ 8 vs. comm parameters.

Setting	Cores	Util.	Latency (μs)
$\kappa=0$ / inter-only	64/64	100%	8.75
$\beta=0.1$	64/64	100%	9.03
$\beta=0.25$	24/64	38%	20.87
$\kappa=\beta=1.0$ (main)	24/64	38%	42.65
Equal-partition 64 cores	64/64	100%	49.48

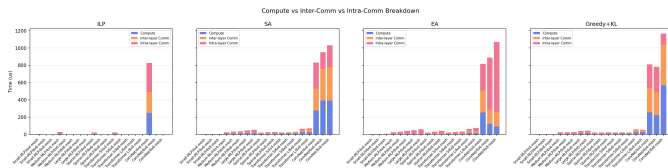


Fig. 3: Compute, inter-layer, and intra-layer latency breakdown.

E. Ablations and Communication Sensitivity

Intra-parallel assumption. Dividing T_{intra} by x_i shifts the ranking to EA 9, SA 7, Greedy+KL 3, removing

Greedy+KL's compactness advantage. κ sweep. Table IV and Fig. 4 show solver wins vs. a global κ multiplier: at $\kappa=0$ SA and Greedy+KL tie (8 each); EA peaks at $\kappa=0.25$ (11); Greedy+KL leads for $\kappa \geq 0.5$ (9–12). Three internally consistent models (main, inter-only, intra-parallel) each crown a different winner.

Communication cost and core utilization. Sweeping β (Table V, Fig. 4) shows Greedy+KL fills 64/64 cores when comm is cheap ($\kappa=0$, $\beta \leq 0.1$) but uses only 24/64 at $\kappa=\beta=1.0$ (Fig. 5); equal 64-core partitioning at default κ is slower (49.5 vs. 42.7 μs).

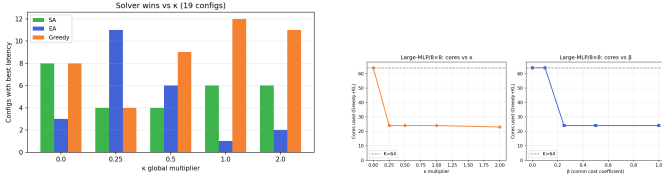


Fig. 4: Left: solver wins vs. κ . Right: Greedy+KL cores used on Large-MLP/8 $\times$ 8 vs. κ and β (dashed = full mesh).

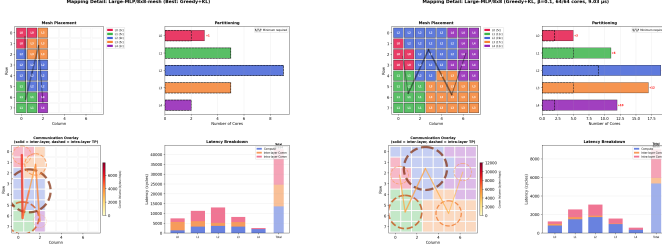


Fig. 5: Greedy+KL on Large-MLP/8 $\times$ 8: main model $\kappa=\beta=1$, 24/64 cores (left, 42.7 μ s) vs. cheap-comm sensitivity $\beta=0.1$, 64/64 cores (right, 9.03 μ s).

VI. Conclusion

We compared four mapping solvers under a corrected setup where idle cores are allowed. Rankings are not robust to the cost model: Greedy+KL leads the main model (12/19), while inter-only, intra-parallel, and κ sweeps each crown a different winner. Limitations: serial execution, inconsistent inter/intra parallelism, linear-chain Transformers, assumed κ, β . Future work: compactness-biased neighborhoods for SA/EA, pipelined objectives, and hardware validation.

References

- [1] J. Yik et al., “Modeling and optimizing performance bottlenecks for neuromorphic accelerators,” arXiv:2511.21549, 2025.
- [2] J. Timcheck et al., “A compute and communication runtime model for Loihi 2,” arXiv:2601.10035, 2026.
- [3] M. Wever et al., “Evolutionary mapping of neural networks to spatial accelerators,” arXiv:2602.04717, 2026.
- [4] A. Parashar et al., “Timeloop,” in ISPASS, 2019, pp. 304–315.
- [5] S. Kao and T. Krishna, “GAMMA,” in ICCAD, 2020, pp. 1–9.
- [6] S. Kao et al., “DiGAMMA,” in DATE, 2022, pp. 232–237.
- [7] S. Lie, “Inside the Cerebras wafer-scale cluster,” IEEE Micro, 2024.
- [8] D. Abts et al., “Think fast: A TSP,” in ISCA, 2020.